

Guardian Shell Engineering Architecture Review

Prepared for engineering review Date: March 13, 2026 Audience: security engineers, platform engineers, systems engineers

Executive Summary

Guardian Shell is an agent-centric Linux security system that combines:

- eBPF tracepoints
- BPF-LSM enforcement
- cgroup-backed agent identity
- userspace policy management
- dashboard, alerting, and approval workflows

It is already a credible architecture for agent governance, but its current trust boundary is not yet as strong as it should be for adversarial LLM-agent behavior.

The primary technical issue is that core policy still depends too heavily on path-based, tracepoint-first reasoning. The right direction is a layered evolution toward:

1. cgroup-first identity
2. LSM-first file and exec decisions
3. stronger executable identity for high-assurance mode
4. seccomp for dangerous syscall classes
5. session-scoped policy state
6. optional stronger isolation tiers for high-risk deployments

Current System Overview

Current implemented capabilities

The current codebase includes:

- sys_enter_openat, openat2, and legacy open tracing
- file_open LSM-based file blocking
- sys_enter_execve tracing
- bprm_check_security LSM-based exec blocking

- sys_enter_connect tracing for outbound network visibility
- sched_process_fork and exit tracking
- cgroup, TGID, and comm-based identity
- cgroup launcher and IPC registration
- alerting, dashboard, metrics, CLI, and permission management

Current architecture in concrete terms

The current Guardian Shell architecture is best described as a hybrid system with four cooperating parts:

1. a launcher and registration path
2. a long-running daemon and policy engine
3. kernel-resident eBPF programs
4. a user-visible operations plane

This is important because Guardian is not only an eBPF program. The security story depends on all four parts working together.

High-level architecture

User space

- guardian daemon
 - config parsing
 - BPF loading
 - BPF map population
 - event processing
 - alerting / dashboard / permissions

- guardian-launch
 - cgroup creation
 - resource-limit setup
 - cgroup registration via IPC
 - move self to cgroup
 - exec target command

Kernel space

- tracepoints
 - open/openat/openat2
 - execve
 - connect
 - fork/exit

LSM hooks

- file_open
- bprm_check_security

BPF maps

- identity maps

- allow/deny policy maps
- pending deny maps
- event buffers

Current architecture by responsibility

1. guardian-launch

This is the secure entry point for high-assurance sessions.

It is responsible for:

- creating the agent cgroup
- setting cgroup controller limits
- deriving the kernel-visible cgroup identity
- registering the session with the daemon
- entering the cgroup before the target command starts

This matters because identity is strongest when it is established before the agent begins executing.

2. guardian daemon

The daemon is the policy and control-plane center.

It is responsible for:

- reading and validating configuration
- loading BPF programs
- attaching tracepoints and LSM hooks
- populating and updating BPF maps
- consuming event streams
- applying user-space policy checks and approval logic
- serving dashboards, alerts, and metrics

This is where Guardian differs from many research-style eBPF tools. The daemon is not incidental. It is part of the product.

3. guardian-ebpf

The kernel side is responsible for:

- identifying the current process or agent session
- reading relevant syscall or hook context

- evaluating kernel-side subsets of policy
- setting pending deny flags where current architecture still uses two-phase enforcement
- blocking in LSM hooks
- sending structured events to user space

4. Supporting control surfaces

These include:

- IPC over a Unix socket
- dashboard routes and database storage
- alert sinks such as JSON, webhook, Slack, email, metrics
- permission request handling

This matters because the product's security value is partly enforcement, but partly decision support and auditability.

Example: what happens when an agent tries to read a secret file

Scenario:

- agent launched through guardian-launch
- policy allows /workspace/project/**
- policy denies /home/dev/.ssh/**
- agent runs `cat /home/dev/.ssh/id_rsa`

Current flow:

1. agent process is already inside its dedicated cgroup
2. tracepoint on open/openat sees the raw requested path
3. kernel code identifies the process by cgroup, TGID, or comm
4. kernel code evaluates current map-based file policy
5. if denied, it writes PENDING_DENY for that pid/tgid
6. file_open LSM hook runs
7. LSM hook sees PENDING_DENY and returns an access error
8. event is emitted to user space
9. daemon logs, alerts, stores, and displays the denial

This example shows both the power and the weakness of the current design:

- power: the access can be blocked in kernel space
- weakness: the core decision was still made from the raw requested path rather than the final kernel-resolved object

Example: what happens when an agent runs a command

Scenario:

- exec policy allows /usr/bin/git
- exec policy denies /usr/bin/curl
- the agent attempts curl https://example.com

Current flow:

1. execve tracepoint captures the requested executable path
2. identity is resolved using cgroup/TGID/comm logic
3. exec policy is checked against kernel maps
4. if denied, PENDING_EXEC_DENY is written
5. bprm_check_security LSM hook runs
6. the pending entry is consumed
7. execution is blocked
8. event is sent to user space for logging and alerting

Again, this is useful today, but weaker than direct binary-identity enforcement.

Agent Identity Design

Why identity is hard

An LLM agent is rarely one stable process with one unique name.

Examples:

- Claude Code may appear as claude and spawn node, bash, git
- Aider may appear as python3
- another Python-based agent may also appear as python3

So identity must solve:

1. distinguishing agents using the same runtime
2. carrying policy across child processes
3. reducing spoofability
4. remaining operationally usable for real developer workflows

Identity layers in the current design

1. comm process name

Pros:

- simple
- good for discovery

- easy fallback mode

Cons:

- 16-byte limit
- ambiguous for common runtimes
- weak against spoofing
- poor fit for subprocess-heavy workloads

Concrete example:

Policy watches: python3

Problem:

```
aider          -> python3
open-interpreter -> python3
custom agent    -> python3
```

Result:

a name-only policy cannot tell these sessions apart

2. TGID and child-process tracking

Pros:

- better process-family coverage
- improves over name-only matching

Cons:

- transient identifiers
- lifecycle bookkeeping
- still weaker than session-backed identity

Concrete example:

```
main agent pid/tgid: 4200
agent forks bash:    4210
bash runs git:       4215
```

If only the original process is watched, git may escape governance.
TGID/child tracking closes part of that gap.

3. Cgroup identity

This is the strongest current identity method.

Pros:

- session-scoped

- inherits automatically to children
- harder to spoof
- aligns with resource governance and lifecycle management

This should be considered the target default secure mode.

Concrete example:

```
guardian-launch --name claudex-fix -- claudex
```

Processes later seen:

```
claudex
bash
git
pytest
python3
```

All stay attributable to the same cgroup-backed agent session.

Cgroup implementation details

1. guardian-launch --name <agent> -- <cmd>
2. create /sys/fs/cgroup/guardian/<agent>-<pid>
3. enable subtree controllers
4. apply memory/pids/cpu limits if configured
5. stat cgroup directory and derive inode-based cgroup ID
6. send IPC registration to daemon
7. daemon updates WATCHED_CGROUPS / ENFORCE_CGROUPS / defaults
8. launcher writes its PID into cgroup.procs
9. launcher execs target command
10. descendants inherit cgroup membership
11. eBPF reads bpf_get_current_cgroup_id()
12. policy is applied using cgroup-first identity

Identity priority in kernel logic

Conceptually:

```
if current cgroup is watched:
    watched
else if current TGID or tracked child is watched:
    watched
else if current comm is watched:
    watched
else:
    ignore
```

Identity issues encountered

1. multiple agents sharing python3 or node
2. child-process escape from name-only policy
3. weak trust in process-name identity

- 4. discovery gaps without explicit launch registration
- 5. operational need to keep the identity model understandable to users

Issues faced in more detail

Issue A: Name collisions among real agents

Problem:

The actual ecosystem of AI tools often reuses generic runtimes.

Examples:

- Aider and a custom Python agent both become python3
- Claude-related helpers may appear as node
- different Electron-based tools may look similar in process listings

Why this matters:

- policy may attach to the wrong process family
- monitoring output becomes hard to interpret
- one agent may accidentally inherit another agent's visibility

Current answer:

- prefer cgroup-backed launch mode
- use comm-based mode only as compatibility or discovery mode

Issue B: The real action often happens in subprocesses

Problem:

The agent prompt loop itself may not directly perform the risky action. Instead, it spawns a shell or helper.

Example:

```
agent prompt process -> bash -> cat ~/.aws/credentials
```

If identity sticks only to the top-level process name, the dangerous access is missed.

Current answer:

- sched fork tracking
- TGID and child maps
- cgroup inheritance as the cleanest solution

Issue C: Discovery after-the-fact is weaker than explicit registration

Problem:

If the daemon has to discover agents only by scanning /proc, then:

- agents started later may be missed for some interval
- attribution is inferential rather than explicit
- race windows become harder to reason about

Current answer:

- rescans still exist as a fallback
- guardian-launch makes the session boundary explicit before agent work starts

Issue D: Identity must work across monitoring and enforcement

Problem:

If the identity model used for logs differs from the model used for blocking, operator trust degrades.

Current answer:

- the kernel identity helpers prioritize the same layered checks used across event types
- cgroup identity is treated as the strongest signal in both monitoring and enforcement paths

Current solution

The present design solves these pragmatically through:

- launcher-managed cgroup sessions
- TGID and child tracking
- comm as compatibility mode rather than strong identity

Why cgroups are the right center of gravity

For LLM agents, the best identity unit is not "the executable" and not "the current process name." It is the agent session.

Cgroups match that need well because:

1. they cover the whole subprocess family
2. they align with resource controls
3. they provide a durable kernel-visible identity
4. they fit lifecycle actions like stop, list, and audit

This is why the future architecture should make cgroup-backed identity the explicit secure default, not just one available option.

Current Enforcement Model

File access flow

```
open/openat/openat2 tracepoint
-> read raw filename
-> identify process
-> evaluate allow/deny maps
-> if denied, write PENDING_DENY
file_open LSM hook
-> consume PENDING_DENY
-> return -EACCES when present
```

Example:

```
Policy:
allow /workspace/project/**
deny  /home/dev/.ssh/**
```

```
Agent action:
open("/home/dev/.ssh/id_rsa")
```

```
Result today:
deny decision is made from the tracepoint-visible path
LSM hook blocks using the pending flag
```

Exec flow

```
execve tracepoint
-> read raw executable path
-> evaluate exec policy
-> if denied, write PENDING_EXEC_DENY
bprm_check_security LSM hook
-> consume PENDING_EXEC_DENY
-> return denial when present
```

Example:

```
Policy:
allow /usr/bin/git
deny  /usr/bin/curl
```

```
Agent action:
execve("/usr/bin/curl", ...)
```

```
Result today:
tracepoint sets pending exec deny
LSM hook blocks execution
```

Why this design exists

The architecture separates:

- tracepoint-friendly capture of syscall arguments
- LSM-based enforcement

This is pragmatic because raw syscall arguments are easier to capture at tracepoints than to reconstruct inside the LSM hook.

It is a reasonable transitional architecture, especially when building a product incrementally. But it is not the ideal end-state architecture for adversarial agent security.

Strengths

1. good observability
2. simple path-policy semantics for current product needs
3. incremental compatibility across kernels and deployments
4. keeps enforcement in kernel while preserving a rich user-space product plane

Why the current architecture was a sensible starting point

From an engineering perspective, the current architecture likely emerged because it optimizes for practical implementation constraints:

1. tracepoints are easier for reading syscall arguments
2. path strings are easier to reason about initially than inode or object identity
3. user-space policy and dashboard work can proceed in parallel with kernel enforcement work
4. fallback behavior is easier when some advanced kernel features are unavailable

This is worth stating explicitly. The current design is not irrational. It is a pragmatic intermediate architecture.

Weaknesses

1. trust boundary remains tracepoint-first for core policy
2. path identity is weak against aliasing
3. pending-map handoff complicates correctness reasoning
4. some policy state is still global rather than fully per session

Why these weaknesses matter more for LLM agents than for ordinary workloads

Traditional workloads usually do not actively search for bypasses.

LLM agents may:

- try alternate file names
- try helper binaries
- try subprocess chains
- try alternate system interfaces
- ask for policy changes when blocked

That means any weak trust anchor is more likely to be stressed in practice.

Comparison With Veto

Guardian Shell and Veto solve different primary problems.

Veto

Primary question:

"Should this binary run at all?"

Core strengths:

- binary identity via content hash
- strong pre-exec trust anchor
- robust against rename, copy, and symlink-based executable bypasses

Guardian Shell

Primary question:

"What can this agent session access?"

Core strengths:

- file governance
- per-agent/session policy
- approval workflow
- observability and operations plane
- better fit for interpreter-heavy agent behavior

Engineering comparison

+-----+-----+-----+			
Dimension	Guardian Shell	Veto	

Primary unit	agent/session	executable binary	
Identity anchor	cgroup/TGID/comm	content hash	
File governance	yes	limited	
Exec governance	path-based today	stronger	
Approval workflow	yes	not core focus	
Observability plane	rich	more enforcement-centric	
Best fit	agent resource governance	strong binary execution trust	

Practical conclusion

Veto is stronger at execution identity.

Guardian Shell is stronger as an agent-governance platform.

For high-assurance deployments, a Veto-like execution trust primitive and Guardian-like resource governance can coexist.

Example scenarios

Scenario 1: copied binary bypass

Denied path:

/usr/bin/curl

Agent action:

cp /usr/bin/curl /tmp/.x
/tmp/.x https://example.com

Veto-style binary identity is stronger here.

Scenario 2: interpreter-heavy task

Allowed binary:

/usr/bin/python3

Actual risk:

python reads secrets from ~/.ssh or ~/.aws

Guardian-style file governance is stronger here.

Scenario 3: human approval workflow

Agent asks for:

/workspace/legacy-migration/**
duration: 20 minutes

Guardian is the more natural product fit because approval and audit are first-class product features.

Current Security Limitations

1. Path aliasing and canonicalization gaps

The main problem is that policy still depends heavily on raw path strings seen at syscall entry.

Examples:

- /proc/self/root/etc/shadow
- symlink indirection
- bind mounts
- hard links

Userspace normalization is useful, but not a full substitute for canonical or inode-aware enforcement.

Example scenario:

Denied:
 /etc/shadow

Agent tries:
 /proc/self/root/etc/shadow

If policy relies on the raw requested string, equivalence with the real target may be incomplete.

2. Two-phase decision and enforcement

The PENDING_DENY and PENDING_EXEC_DENY handoff model adds:

- timing assumptions
- cleanup requirements
- harder reasoning across edge cases

Example scenario:

tracepoint decides deny
LSM later consumes pending state

Any architecture that splits "decide" and "enforce" must reason carefully about races, cleanup, and hook ordering assumptions.

3. Exec policy is path-based, not binary-identity-based

This is materially weaker against adaptive agents.

Example scenario:

Denied:

```
/usr/bin/wget
```

Agent:

```
cp /usr/bin/wget /tmp/helper  
/tmp/helper
```

4. Interpreters compress many actions behind one allowed binary

Allowing python3 or node is not meaningful by itself unless file and resource governance is strong underneath it.

Example scenario:

Allowed:

```
/usr/bin/python3
```

But script reads:

```
~/.aws/credentials  
~/.ssh/id_rsa
```

5. Network policy is behind file and exec maturity

Outbound connect visibility exists, but enforcement depth is not yet at the level many security teams will expect.

Example scenario:

Agent cannot read a local secret file

Agent instead tries to download a remote helper or exfiltrate data over HTTPS

6. Alternate surfaces such as io_uring

Tracepoint-centric observability must always account for alternate interfaces that do not pass through the same path.

Example scenario:

Expected observation path:

```
regular syscall tracepoint
```

Alternate path:

```
io_uring-based operation
```

7. Shared host-kernel trust model

Even a strong BPF-based design is still a host-level control model, not the same as VM isolation.

Example scenario:

High-risk third-party agent plugin runs on the same host kernel as developer workloads.

This is where stronger isolation tiers become necessary.

Summary of limitations

The current architecture is good enough to demonstrate product value and meaningful policy enforcement, but it is not yet the final form needed for stronger adversarial-agent claims.

Recommended Future Architecture

The correct future design is an evolution, not a rewrite.

Design principles

1. keep the existing user-space product surface
2. strengthen the kernel trust anchor
3. make cgroup-backed session identity primary
4. separate agent identity from resource identity
5. support multiple assurance tiers

Proposed layered architecture

Product layer

- dashboard
- alerts
- approvals
- audit trail
- policy authoring

Agent identity layer

- cgroup as primary session identity
- launcher-managed lifecycle
- per-agent policy object

Kernel enforcement layer

- LSM file_open with canonical or inode-aware decisions
- LSM bprm_check_security with stronger exec identity
- seccomp for dangerous syscalls
- network enforcement growth over time

Host / isolation layer

- AppArmor or SELinux baseline
- optional namespace/container isolation
- optional microVM / VM high-assurance tier

Why the proposed architecture is better in engineering terms

The proposed architecture is better not because it is more complex, but because it moves each decision to the layer that can evaluate it more correctly.

Examples:

- file identity decisions move closer to the real file object
- dangerous syscall classes move to seccomp
- agent identity is anchored to cgroups rather than names
- observability stays rich, but is no longer the same thing as the trust anchor

Current vs proposed

Area	Current Guardian Shell	Proposed Future Guardian Shell
Primary file decision	tracepoint-first, path string	LSM-first, canonical/inode-aware
File blocking	pending deny handoff	decision and block in one place
Exec control	path-based exec policy	stronger executable identity
Agent identity	cgroup/TGID/comm	cgroup-first
Network	connect monitoring	policy-aware enforcement
Dangerous syscalls	limited	seccomp standard layer
Policy isolation	partly global maps	per-session policy state

Proposed file flow

Agent open()

- > LSM file_open receives real file object
- > resolve canonical path or inode/device identity
- > evaluate per-agent rules directly in the LSM path
- > allow or deny
- > emit observability event

Example scenario:

Denied:

/home/dev/.ssh/**

Agent tries:

/tmp/key-link -> /home/dev/.ssh/id_rsa
open("/tmp/key-link")

In the proposed design, the file decision should be made against the resolved target identity rather than only the requested string.

Proposed exec flow

Agent execve()

- > LSM bprm_check_security
- > resolve executable identity
- > standard mode: canonical path policy
- > high-assurance mode: binary identity / measured identity
- > allow or deny
- > emit observability event

Example scenario:

Denied executable identity:
curl binary

Agent tries:
copy binary to /tmp/.curl2
exec that copy

In a high-assurance exec mode, the copied binary should still be identified as the same executable identity.

Why this is better for LLM agents

1. LLM agents are adaptive and try alternative approaches
2. LLM agents are process ecosystems rather than single binaries
3. LLM agents need dynamic policy and temporary approvals
4. agent security needs both prevention and explanation
5. deployment risk varies, so progressive assurance matters

Expanded explanation: why this architecture is better for LLM agents

1. It assumes the agent is workaround-seeking

A normal process often just fails when denied.

An LLM agent may:

- search docs
- try another path
- try another process
- try another interface
- ask for the control to be disabled

The proposed design is better because it reduces the number of weak intermediate representations the agent can exploit.

2. It models the agent as a session, not just an executable

This is a better fit for:

- shell-based workflows
- interpreter-heavy workflows
- subprocess trees
- temporary human approvals

3. It preserves product usability while strengthening enforcement

This is a critical difference from simply pushing everything into a static MAC profile.

The proposed design keeps:

- approvals
- audit
- dashboards
- event context

while strengthening the kernel trust anchor underneath them.

Key implementation improvements

Improvement 1: move file decisions into file_open

This is the highest-value change.

Why:

- the hook sees the real file object
- canonical or inode-aware reasoning becomes possible
- the pending-handoff model can be reduced or eliminated for core file policy

Implementation direction:

1. keep tracepoints for logging
2. move primary file allow/deny into LSM logic
3. add inode-aware exact-deny support for high-value resources

Improvement 2: strengthen exec identity

Recommended modes:

1. standard path mode
2. high-assurance binary-identity mode

Why:

- path-based exec policy is the wrong long-term trust anchor for adversarial agents
- binary identity directly addresses rename/copy evasions

Engineering note:

The implementation may need tiered support depending on kernel features, IMA availability, or helper support.

Improvement 3: make policy state session-scoped

Conceptual target:

AGENT_SESSIONS:

cgroup_id -> agent_id, generation_id, enforcement_flags

FILE_POLICY:

(agent_id, generation_id, resource_key) -> allow/deny

EXEC_POLICY:

(agent_id, generation_id, exec_key) -> allow/deny

TEMP_GRANTS:

(agent_id, resource_key, expiry) -> grant metadata

Why:

- clearer reasoning
- less cross-agent coupling
- cleaner reload semantics
- easier answer to: "why was this allowed?"

Improvement 4: add seccomp as standard defense-in-depth

Block or heavily review:

- ptrace
- bpf
- mount
- setns
- unshare
- io_uring_setup

Why:

These are escape-option reducers. They shrink the search space for workaround-seeking agents.

Improvement 5: mature the network layer

Target:

- cgroup-scoped egress policy
- DNS/connect correlation
- special handling for metadata services

Example scenarios:

1. block all direct metadata-service access
2. allow only corporate artifact mirrors
3. allow outbound HTTPS only for approved package hosts

Failure-model expectations

The design should explicitly define whether subsystems fail:

- open
- closed
- monitor-only

Suggested direction:

- dashboard failure should not affect enforcement
- alerting failure should not affect enforcement
- missing advanced hook support should produce explicit downgrade signals

This is important for trust. Silent security downgrades are product failures even when they are technically graceful.

Testing strategy

The architecture should be tested against agent-like adversarial behavior, not only syscall correctness.

Key test categories:

1. path alias tests
2. exec identity tests
3. session identity tests
4. policy isolation tests
5. bypass-surface tests such as io_uring

Expanded examples:

1. path alias tests
 - /proc/self/root - symlink chains - hard-link aliasing - bind mounts
2. exec identity tests
 - copy binary - rename binary - wrapper-loader execution
3. identity tests

- launcher session registration - child-process inheritance - stale cgroup cleanup

4. multi-agent tests

- two Python agents with conflicting policy - concurrent grants and reloads

5. exfiltration tests

- blocked local file access followed by remote fallback attempt

Example deployment scenarios

Scenario A: developer workstation coding assistant

Best fit:

- cgroup launch mode
- file policy
- exec policy
- approvals
- monitoring plus selective enforcement

Why:

High usability is required, but governance still matters.

Scenario B: CI repair agent

Best fit:

- cgroup identity
- stronger default-deny
- reduced network egress
- seccomp enabled

Why:

The environment is more controlled, so stronger defaults are acceptable.

Scenario C: infrastructure-affecting agent

Best fit:

- stronger exec identity
- strong network restrictions
- baseline AppArmor/SELinux

- optional isolated runtime tier

Why:

Impact radius is larger, so shared-host trust may be insufficient.

Technical Primer

What is eBPF?

eBPF lets verified programs run inside the Linux kernel.

What is the verifier?

The verifier proves safety properties such as bounded execution and safe memory access.

What are tracepoints?

Kernel instrumentation points used for syscall and scheduler event capture.

What are LSM hooks?

Kernel security decision points such as `file_open` and `bprm_check_security`.

What is BPF-LSM?

Attaching eBPF programs to LSM hooks.

What are BPF maps?

Shared kernel/user-space key-value structures used for:

- watched identities
- enforcement identities
- allow/deny rules
- pending decisions
- event transport support

What is IPC?

Inter-process communication.

In Guardian Shell it is primarily used for local coordination between:

- guardian-launch
- guardian
- control utilities

What is a Unix socket?

The local IPC transport endpoint used by the control plane.

What is a perf event array?

The mechanism used to send structured events from kernel space to user space.

What is a cgroup?

A kernel process-grouping mechanism used here for both resource control and session identity.

What are TGID and comm?

- TGID: process-family identifier
- comm: short process name

Both are useful, but weaker than cgroup-backed identity.

What are openat, execve, and connect?

Core syscalls for:

- file access
- program execution
- outbound network connection

What is TOCTOU?

Time-of-check to time-of-use. Relevant whenever a decision is made in one kernel context and consumed later.

What is canonical path resolution?

Resolving the real target path after symlink, traversal, and mount effects.

What is an inode?

The kernel's file object metadata identity. Often a stronger basis than names alone.

Final Engineering Assessment

Guardian Shell has the right product shape for LLM-agent governance, but its security core still needs to evolve toward stronger identity and stronger enforcement locality.

The most important engineering decisions are:

1. make cgroup-backed session identity primary

2. move file and exec decisions deeper into LSM-first logic
3. add seccomp as standard hardening
4. isolate policy per session
5. preserve the current rich observability and approval plane

If those changes are made, the architecture will be substantially better aligned with the actual LLM-agent threat model.

References

Internal material used:

- docs/architecture-analysis.md
- docs/guardian-shell-vs-veto-comparison.md
- docs/security-improvements-research.md
- docs/sandboxing-deep-dive.md
- docs/technical-comparison.md
- AGENT_IDENTITY.md
- implementation in guardian-ebpf/src/main.rs, guardian/src/main.rs, guardian/src/config.rs, and guardian-launch/src/main.rs

External references checked:

- Ona, "Introducing Veto: security for the next era of software"
<https://ona.com/stories/introducing-veto-security-for-the-next-era-of-software>
- Linux kernel docs, "LSM BPF Programs"
https://docs.kernel.org/bpf/prog_lsm.html
- eBPF Docs, bpf_d_path
https://docs.ebpf.io/linux/helper-function/bpf_d_path/
- Linux kernel docs, "Landlock"
<https://docs.kernel.org/userspace-api/landlock.html>
- Ubuntu documentation, "AppArmor"
<https://ubuntu.com/server/docs/how-to/security/apparmor/>
- Red Hat documentation, "Using SELinux"

https://docs.redhat.com/en/documentation/red_hat_enterprise_linux/10/html-single/using_selinux/using_selinux